

贵州大学
GUIZHOU UNIVERSITY

实践（实验）报告

课程名称： Java Web 后端开发技术

学　　院： 示范性软件工程学院

授课教师： 丁玺润

专业班级： 软件工程 2403

学　　号： 2400770225

姓　　名： 皮宸西

小组成员： 皮宸西

2026 年 9 月 17 日

实验名称：	实验二：Controller 接口开发与参数接收

一、实验目的：

1. 掌握 @RestController 与 @RequestMapping 系列注解的使用，理解请求映射机制。
2. 掌握 @RequestParam、@PathVariable、@RequestBody 三种参数接收方式及其适用场景。
3. 掌握 JSON 响应数据的返回方式，理解前后端分离下的数据交互格式。

二、实验环境：

硬件环境：PC 计算机（内存 8GB 及以上）。软件环境：Windows 11、JDK 17.0.20.1（Temurin）、Maven 3.9.16、IntelliJ IDEA 2025.3、Spring Boot 3.3.5、MySQL 8.0.29、内嵌 Tomcat 10.1.31；接口调试使用 curl 命令行 HTTP 客户端（与 Apifox、Postman 等 REST 客户端等价）。前置条件：已完成实验一的 CTTS 项目搭建。

三、实验内容：

1. 编写 Controller 层，实现"查询实训项目列表"接口（先用模拟数据返回 JSON）。
2. 实现按 id 查询单个实训项目详情接口（@PathVariable）。
3. 实现分页查询接口（@RequestParam 接收页码、每页条数）。
4. 使用 Apifox 对各接口进行调试，验证参数接收与 JSON 返回。

四、实验步骤（包含运行结果截图）：

（1）梳理工程结构，建立 Controller 层

实验一已完成 CTTS 工程骨架（`com.gzu:ctts`，Spring Boot 3.3.5，JDK 17，Maven 构建）。实验二的任务是在该骨架上补齐接口层，因此先按「接口层—业务层—模型层」的职责划分建立包结构，各包的职责见表 2.1。

表 2.1 实验二新增的包及职责

包	主要类	职责
controller	ProjectController	接收请求、绑定参数、返回 JSON
service	ProjectService	业务逻辑接口

service.impl	ProjectServiceImpl	业务逻辑实现，本实验用内存模拟数据占位
entity	Project	领域对象，字段与数据库表一一对应
dto	Result、PageResult、ProjectForm	统一响应、分页结果、新增请求体
exception	BizException、GlobalExceptionHandler	业务异常与全局异常处理器

需要强调：Controller 只负责「协议转换」，即把 HTTP 报文翻译成 Java 方法调用、再把返回值翻译成 JSON；真正的业务规则放在 Service。这样划分后，后续把模拟数据换成 MyBatis 数据库查询时，Controller 代码无需改动。

新建后的工程目录结构如图 2.1 所示。

图 2.1 项目目录结构（实验二新增 Controller 层）

```
C:\Users\asus\OneDrive\Desktop\javaweb\实验2\ctts> dir /s /b src

src
  main
    java
      com
        gzu
          ctts
            CttsApplication.java
            controller
              ProjectController.java
            dto
              PageResult.java
              ProjectForm.java
              Result.java
            entity
              Project.java
            exception
              BizException.java
              GlobalExceptionHandler.java
            service
              ProjectService.java
              impl
                ProjectServiceImpl.java
    resources
      application.yml
  test
    java
      com
        gzu
          ctts
            CttsApplicationTests.java

说明：实验二在实验一骨架上新增 controller / service / service.impl /
entity / dto / exception 六个包，共 10 个 Java 源文件。
```

图 2.1 CTTS 工程目录结构（实验二新增 Controller 层）

（2）编写统一响应封装 Result

前后端分离场景下，服务端固定返回「状态码 + 提示信息 + 业务数据」三段式 JSON，前端只需判断 `code` 就能区分成功与失败，不必再解析 HTTP 状态码。`Result<T>` 用泛型承载业务数据，并提供 `ok()`、`fail()` 静态工厂方法，使 Controller 的返回语句保持简洁。源码如图 2.2 所示。

Result.java（第 1–62 行 / 共 62 行）

```java
package com.gzu.ctts.dto;

import lombok.Data;

import java.io.Serializable;

/**
 * 统一响应结果封装。
 *
 * <p>前后端分离场景下，服务端固定返回「状态码 + 提示信息 + 业务数据」三段式 JSON，
 * 前端只需判断 {@code code} 即可区分成功与失败，无需再解析 HTTP 状态码：
 * <pre>
 * { "code": 200, "message": "success", "data": { ... } }
 * </pre>
 *
 * <p>泛型 {@code T} 表示业务数据类型，配合静态工厂方法 {@code ok()} / {@code fail()}
 * 可以让 Controller 的返回语句保持简洁。
 *
 * @param <T> 业务数据类型
 */
@Data
public class Result<T> implements Serializable {

    /** 成功状态码 */
    public static final int CODE_SUCCESS = 200;

    /** 参数错误状态码 */
    public static final int CODE_BAD_REQUEST = 400;

    /** 资源不存在状态码 */
    public static final int CODE_NOT_FOUND = 404;

    /** 业务状态码 */
    private Integer code;

    /** 提示信息 */
    private String message;

    /** 业务数据 */
    private T data;

    private Result(Integer code, String message, T data) {
        this.code = code;
        this.message = message;
        this.data = data;
    }

    /** 成功，带数据 */
    public static <T> Result<T> ok(T data) {
        return new Result<>(CODE_SUCCESS, "success", data);
    }

    /** 成功，带数据与自定义提示 */
    public static <T> Result<T> ok(String message, T data) {
        return new Result<>(CODE_SUCCESS, message, data);
    }

    /** 失败，自定义状态码与提示 */
    public static <T> Result<T> fail(Integer code, String message) {
        return new Result<>(code, message, null);
    }
}
```

图 2.2 统一响应封装 Result.java

其中 `code` 沿用 HTTP 语义（200 成功、400 参数错误、404 资源不存在），`data` 为泛型字段，既能装单个对象，也能装集合或分页对象。

（3）编写请求参数对象 ProjectForm

`@RequestBody` 需要一个与 JSON 结构对应的 Java 类。`ProjectForm` 除承载字段外，还用 `@NotBlank`、`@Size` 声明校验规则；配合 Controller 参数上的 `@Valid`，非法请求会在进入业务逻辑之前就被拦截。源码如图 2.3 所示。

ProjectForm.java (第 1–32 行 / 共 32 行)

```java
package com.gzu.ctts.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Data;

/**
 * 新增实训项目的请求参数对象（DTO）。
 *
 * <p>用于演示 {@code @RequestBody} 接收 JSON 请求体：
 * Spring MVC 借助 Jackson 把请求体里的 JSON 反序列化成该对象，
 * 再配合 {@code @Valid} 触发 JSR-380 参数校验。
 */
@Data
public class ProjectForm {

    @NotBlank(message = "项目名称不能为空")
    @Size(max = 100, message = "项目名称长度不能超过 100 个字符")
    private String name;

    private String category;

    private String teacher;

    private String deadline;

    /** 状态：0-未开始 1-进行中 2-已完成，缺省为 0 */
    private Integer status = 0;

    @Size(max = 500, message = "项目描述长度不能超过 500 个字符")
    private String description;
}
```

图 2.3 新增请求参数对象 ProjectForm.java

（4）编写 Service：先用模拟数据返回

按实验内容要求，本阶段的接口先返回模拟数据，不接入数据库。`ProjectServiceImpl` 用 `ArrayList` 维护 8 条实训项目记录，对外提供四个方法，见表 2.2。数据在应用重启后重置，因此与实验一建立的 `ctts` 库互不干扰。

表 2.2 ProjectService 提供的方法

方法	说明
`listAll()`	返回全部实训项目

`getById(Integer id)`	按 id 查找，找不到返回 `null`
`page(int pageNum, int pageSize)`	按页码切片，返回 `PageResult`（含当前页数据、总数、总页数）
`create(ProjectForm form)`	追加一条记录，id 取当前最大值 +1

（5）编写 ProjectController，实现三类参数接收

控制器类上的 `@RestController` 等价于 `@Controller` 与 `@ResponseBody` 的组合：既把本类注册为 Spring MVC 的处理器，又让所有方法的返回值经 `HttpMessageConverter`（本项目是 Jackson）转成 JSON 写入响应体，而不是当成视图名去解析模板。类上的 `@RequestMapping("/api/projects")` 提供统一路径前缀，方法级注解只需写相对路径；类上的 `@Validated` 则开启方法级参数校验，使 `@RequestParam`、`@PathVariable` 上的 `@Min`、`@Max` 生效。控制器完整源码见图 2.4 至图 2.6。

ProjectController.java（第 1–54 行 / 共 160 行）

```java
package com.gzu.ctts.controller;

import com.gzu.ctts.dto.PageResult;
import com.gzu.ctts.dto.ProjectForm;
import com.gzu.ctts.dto.Result;
import com.gzu.ctts.entity.Project;
import com.gzu.ctts.exception.BizException;
import com.gzu.ctts.service.ProjectService;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * 实训项目 Controller —— 实验二核心类。
 *
 * <p>负责把 HTTP 请求映射到具体方法，并把 Service 返回的业务对象序列化成 JSON。
 * 本类集中演示三类参数接收方式：
 * <ol>
 *   <li>{@code @RequestParam}：接收查询串参数，如 {@code ?pageNum=1&pageSize=3}；</li>
 *   <li>{@code @PathVariable}：接收 URL 路径占位符，如 {@code /api/projects/3}；</li>
 *   <li>{@code @RequestBody}：接收请求体中的 JSON 并反序列化为对象。</li>
 * </ol>
 *
 * <p>注意类上的 {@code @RestController} 是 {@code @Controller} 与 {@code @ResponseBody}
 * 的组合注解：前者把本类注册为 Spring MVC 的处理器，后者让所有方法的返回值
 * 都经 HttpMessageConverter（这里是 Jackson）转成 JSON 写入响应体，
 * 而不是当成视图名去解析模板。
 *
 * <p>类上的 {@code @Validated} 开启方法级参数校验，
 * 使 {@code @RequestParam}/{@code @PathVariable} 上的 {@code @Min}、{@code @Max} 生效。
 */
@RestController
@RequestMapping("/api/projects")
@Validated
public class ProjectController {

    private static final Logger log = LoggerFactory.getLogger(ProjectController.class);

    private final ProjectService projectService;

    /**
     * 构造器注入。相比字段注入 {@code @Autowired}，
     * 构造器注入能保证依赖不为 null、便于单元测试，也是 Spring 官方推荐的方式。
     */
    public ProjectController(ProjectService projectService) {
        this.projectService = projectService;
    }

```

图 2.4 ProjectController.java（第 1 – 54 行：类声明、构造器注入与接口 1）

ProjectController.java（第 55–107 行 / 共 160 行）

```
55      /**
56       * 接口 1：查询实训项目列表（模拟数据）。
57       *
58       * <pre>
59       * GET /api/projects
60       * </pre>
61       *
62       * <p>路径上没有任何占位符，也没有查询参数，属于最简形式的请求映射。
63       * 返回值是 {@code List<Project>}，Jackson 会把它序列化成 JSON 数组。
64       */
65      @GetMapping
66      public Result<List<Project>> list() {
67          List<Project> projects = projectService.listAll();
68          log.info("查询实训项目列表，共 {} 条", projects.size());
69          return Result.ok(projects);
70      }
71
72      /**
73       * 接口 2：按 id 查询实训项目详情。
74       *
75       * <pre>
76       * GET /api/projects/{id}      例如：GET /api/projects/3
77       * </pre>
78       *
79       * <p>{@code @PathVariable} 取出 URL 中的路径变量 {@code {id}}，
80       * 并按方法参数类型 {@code Integer} 自动完成字符串到数字的类型转换；
81       * 若传入非数字（如 {@code /api/projects/abc}），
82       * Spring 会抛 {@code MethodArgumentTypeMismatchException}，由全局异常处理器兜底。
83       *
84       * <p>{@code @PathVariable("id")} 中的名字必须与 {@code @GetMapping} 里的占位符一致，
85       * 方法参数名与占位符同名时该值可省略。
86       */
87      @GetMapping("/{id}")
88      public Result<Project> detail(@PathVariable("id") Integer id) {
89          Project project = projectService.getById(id);
90          if (project == null) {
91              // 抛出业务异常，由 GlobalExceptionHandler 统一转成 404 的 JSON 响应
92              throw new BizException(Result.CODE_NOT_FOUND, "实训项目不存在，id = " + id);
93          }
94          log.info("查询实训项目详情，id = {}", id);
95          return Result.ok(project);
96      }
97
98      /**
99       * 接口 3：分页查询实训项目。
100      *
101      * <pre>
102      * GET /api/projects/page?pageNum=2&amp;pageSize=3
103      * </pre>
104      *
105      * <p>{@code @RequestParam} 用于接收查询串（query string）中的参数：
106      * <ul>
107      *   <li>{@code required = false} + {@code defaultValue} 让参数可缺省，
```

图 2.5 ProjectController.java（第 55–107 行：接口 2 与接口 3）

ProjectController.java（第 108–160 行 / 共 160 行）

```
108         *       不传时使用默认值，避免「Required request parameter is not present」错误；</li>
109         *   <li>配合类上的 {@code @Validated}，{@code @Min}/{@code @Max} 会对取值做范围校验，
110         *       例如 {@code pageSize=999} 会被拦截并返回 400。</li>
111         * </ul>
112         *
113         * <p>返回值封装成 {@code Result<PageResult<Project>>}：外层是统一响应，
114         * 内层 {@code PageResult} 携带当前页数据与总数、总页数等分页元信息。
115         */
116        @GetMapping("/page")
117        public Result<PageResult<Project>> page(
118                @RequestParam(value = "pageNum", required = false, defaultValue = "1")
119                @Min(value = 1, message = "页码必须大于等于 1") Integer pageNum,
120
121                @RequestParam(value = "pageSize", required = false, defaultValue = "3")
122                @Min(value = 1, message = "每页条数必须大于等于 1")
123                @Max(value = 50, message = "每页条数不能超过 50") Integer pageSize) {
124
125            PageResult<Project> pageResult = projectService.page(pageNum, pageSize);
126            log.info("分页查询实训项目：pageNum = {}，pageSize = {}，命中 {} 条",
127                    pageNum, pageSize, pageResult.getRecords().size());
128            return Result.ok(pageResult);
129        }
130
131        /**
132         * 接口 4：新增实训项目。
133         *
134         * <pre>
135         * POST /api/projects
136         * Content-Type: application/json
137         * Authorization: Bearer &lt;token&gt;
138         *
139         * { "name": "xxx", "category": "Web 后端", ... }
140         * </pre>
141         *
142         * <p>{@code @RequestBody} 告诉 Spring：请求体不是表单键值对，
143         * 而是 JSON，请用 Jackson 反序列化成 {@code ProjectForm} 对象。
144         * 它常与 {@code @Valid} 搭配，触发 DTO 字段上声明的 {@code @NotBlank}、
145         * {@code @Size} 等约束；校验不通过时抛 {@code MethodArgumentNotValidException}。
146         *
147         * <p>{@code @RequestHeader} 用于读取请求头，这里取出 {@code Authorization} 令牌，
148         * 演示「请求头参数」的接收方式。设 {@code required = false} 是让本实验调试更方便，
149         * 真实项目中应设为 true 并由拦截器统一校验。
150         */
151        @PostMapping
152        public Result<Project> create(@Valid @RequestBody ProjectForm form,
153                                      @RequestHeader(value = "Authorization", required = false)
154                                      String token) {
155            log.info("接收新增实训项目请求，Authorization = {}", token);
156            Project created = projectService.create(form);
157            log.info("新增成功，生成的 id = {}", created.getId());
158            return Result.ok("新增实训项目成功", created);
159        }
160    }
```

图 2.6 ProjectController.java（第 108–160 行：接口 4 与请求头参数）

四个接口的清单见表 2.3。四个接口统一用 `Result` 包装返回：接口 1 的数据部分是项目列表，接口 2 与接口 4 是单个项目对象，接口 3 是带分页元信息的 `PageResult`。

表 2.3 实训项目接口清单

接口	请求方式与路径	参数接收方式
接口 1	`GET /api/projects`	无参数
接口 2	`GET`	`@PathVariable`

	/api/projects/{id}	
接口 3	GET /api/projects/page?pageNum=1&pageSize=3	@RequestParam
接口 4	POST /api/projects	@RequestBody、@RequestHeader

三种参数接收方式的适用场景与要点对比见表 2.4。

表 2.4 三种参数接收方式对比

注解	数据来源	典型写法	关键点
@RequestParam	URL 查询串	?pageNum=1&pageSize=3	可设 required=false 与默认值；配 @Min、@Max 校验
@PathVariable	URL 路径占位符	/api/projects/3	名字须与占位符一致；自动做类型转换
@RequestBody	请求体 JSON	{"name":"..."}	需 application/json；配 @Valid 触发校验

（6）构建并启动项目

在 IntelliJ IDEA 中打开工程后，把 Project SDK 指定为 JDK 17、Maven 指向本机 3.9.16（已配置阿里云镜像），并添加一个 Application 运行配置指向 com.gzu.ctts.CttsApplication（程序参数 --server.port=8080），即可在 IDE 内一键运行与断点调试。下面给出等价的命令行方式：先打包生成可执行 fat-jar，再启动应用。

```
mvn clean package -DskipTests
java -jar target/ctts-0.0.1-SNAPSHOT.jar --server.port=8080
```

启动日志如图 2.7 所示。可以看到 Spring Boot 版本为 3.3.5，运行在 Java 17 上，内嵌 Tomcat 10.1.31 监听 8080 端口，整个启动耗时约 3.5 秒。

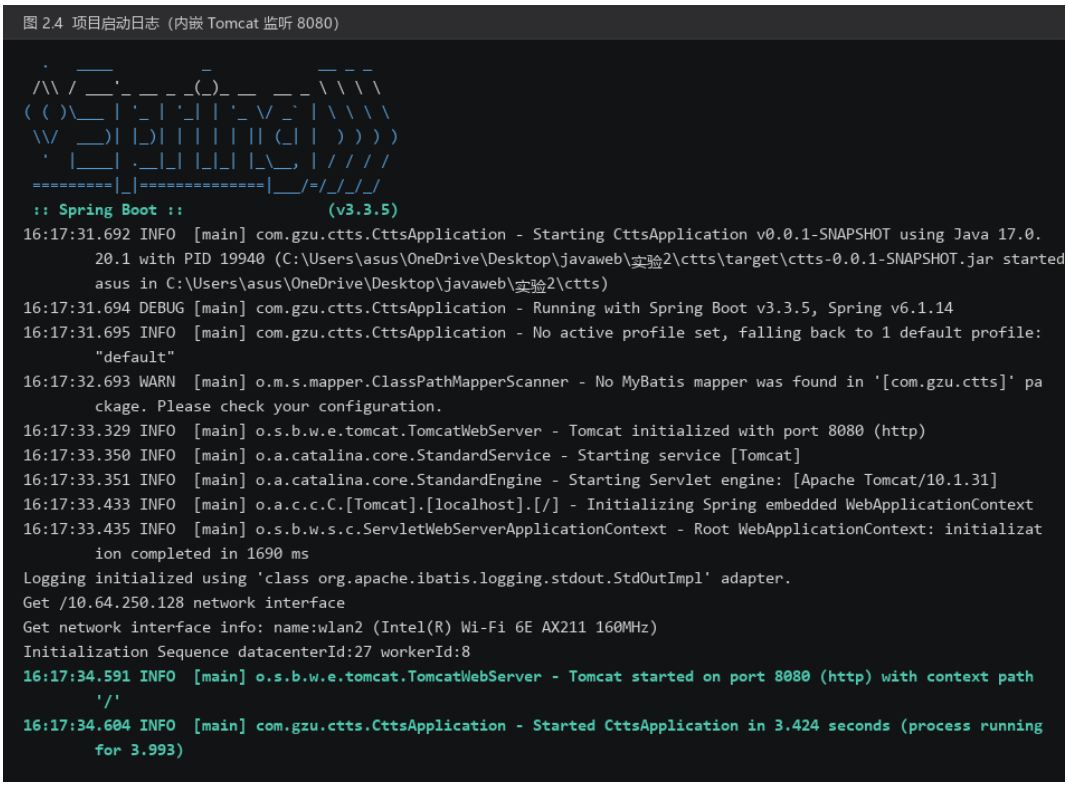


图 2.7 Spring Boot 启动日志

日志中的 `No MyBatis mapper was found in '[com.gzu.ctts]'` 是一条警告而非错误：本实验的接口返回模拟数据，尚未编写 Mapper 接口，因此 MyBatis-Plus 扫描不到任何映射器，属于预期现象，后续实验接入数据库后该警告自动消失。

需说明一点：启动命令显式加了 `--server.port=8080`。原因是 Spring Boot 的配置读取优先级为「命令行参数 > 操作系统环境变量 > 配置文件」，本机环境中存在名为 `SERVER_PORT` 的环境变量，会被宽松绑定规则识别为 `server.port` 而覆盖 `application.yml` 的配置，用命令行参数可以稳定压过它。

（7）使用 HTTP 客户端调试各接口

下面按接口顺序逐个发起真实请求，并保留原始响应报文。

① 查询实训项目列表。路径上既无占位符也无查询参数，是最简形式的请求映射，返回 JSON 数组，如图 2.8 所示。



图 2.8 接口 1 查询实训项目列表——请求与响应

② 按 id 查询详情。`@PathVariable` 取出 URL 中的 `{id}` 并按方法参数类型 `Integer` 自动完成字符串到数字的转换。正常请求（id=3）与两个异常分支的结果如图 2.9 所示：当 id 不存在时，Service 返回 `null`，Controller 主动抛出 `BizException`，由全局异常处理器转成 `code=404` 的 JSON；当传入非数字时，Spring 在参数绑定期就抛出 `MethodArgumentTypeMismatchException`，被统一归一为 `code=400`。

图 2.8 接口2 按 id 查详情（含异常分支）

② 按 id 查询项目详情（@PathVariable，id=3）

```
$ curl -s -i 'http://localhost:8080/api/projects/3'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": {
    "id": 3,
    "name": "实训成果统计报表",
    "category": "数据分析",
    "teacher": "李敏",
    "deadline": "2026-11-30",
    "status": 0,
    "description": "按班级、项目维度统计实训提交率与平均得分。"
  }
}
```

② 异常 项目不存在（id=999）

```
$ curl -s -i 'http://localhost:8080/api/projects/999'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 404,
  "message": "实训项目不存在，id = 999",
  "data": null
}
```

② 异常 路径变量类型不匹配（id=abc）

```
$ curl -s -i 'http://localhost:8080/api/projects/abc'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 400,
  "message": "参数 id 类型不正确，期望 Integer，实际收到：'abc'",
  "data": null
}
```

图 2.9 接口 2 按 id 查详情（含异常分支）

③ 分页查询。`@RequestParam` 接收查询串中的 `pageNum` 与 `pageSize`，两个参数都设了 `defaultValue`，不传时分别取 1 和 3；返回体中的 `PageResult` 除当前页数据外还携带 `total`（总记录数）与 `pages`（总页数），前端据此渲染页码条。第 1 页与第 2 页的响应如图 2.10 所示，可以验证 `total=8`、`pageSize=3` 时 `pages=3`，且两页数据无重叠。

图 2.10 接口3 分页查询（@RequestParam 接收页码与每页条数）

③ 分页查询 第1页 每页3条 (@RequestParam)

```
$ curl -s -i 'http://localhost:8080/api/projects/page?pageNum=1&pageSize=3'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": {
    "records": [
      {"id": 1, "name": "校园实训任务管理系统（CTTS）后端", "category": "Web 后端", "teacher": "丁玺润",
        "deadline": "2026-12-31", "status": 1, "description": "基于 Spring Boot 3 构建实训任务发布、提交与评分接口。"},
      {"id": 2, "name": "学生信息管理模块", "category": "Web 后端", "teacher": "丁玺润", "deadline": "2026-11-15",
        "status": 1, "description": "实现学生信息的增删改查接口，并完成参数校验。"},
      {"id": 3, "name": "实训成果统计报表", "category": "数据分析", "teacher": "李敏", "deadline": "2026-11-30",
        "status": 0, "description": "按班级、项目维度统计实训提交率与平均得分。"}
    ],
    "total": 8,
    "pageNum": 1,
    "pageSize": 3,
    "pages": 3
  }
}
```

③ 分页查询 第2页 每页3条

```
$ curl -s -i 'http://localhost:8080/api/projects/page?pageNum=2&pageSize=3'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": {
    "records": [
      {"id": 4, "name": "前后端分离接口联调", "category": "Web 后端", "teacher": "丁玺润", "deadline": "2026-12-10",
        "status": 1, "description": "使用 Apifox 对全部 REST 接口进行调试与文档生成。"},
      {"id": 5, "name": "任务提交记录导出", "category": "工具开发", "teacher": "王强", "deadline": "2026-12-20",
        "status": 0, "description": "支持将学生提交记录导出为 Excel 文件。"},
      {"id": 6, "name": "登录鉴权与 Token 校验", "category": "Web 后端", "teacher": "李敏", "deadline": "2026-12-05",
        "status": 2, "description": "基于拦截器实现 JWT 登录鉴权与接口权限控制。"}
    ],
    "total": 8,
    "pageNum": 2,
    "pageSize": 3,
    "pages": 3
  }
}
```

图 2.10 接口 3 分页查询（@RequestParam 接收页码与每页条数）

不传任何参数时走默认值；`pageSize` 超过 `@Max(50)` 上限时，方法级校验拦截并返回 `code=400`，两种情形如图 2.11 所示。

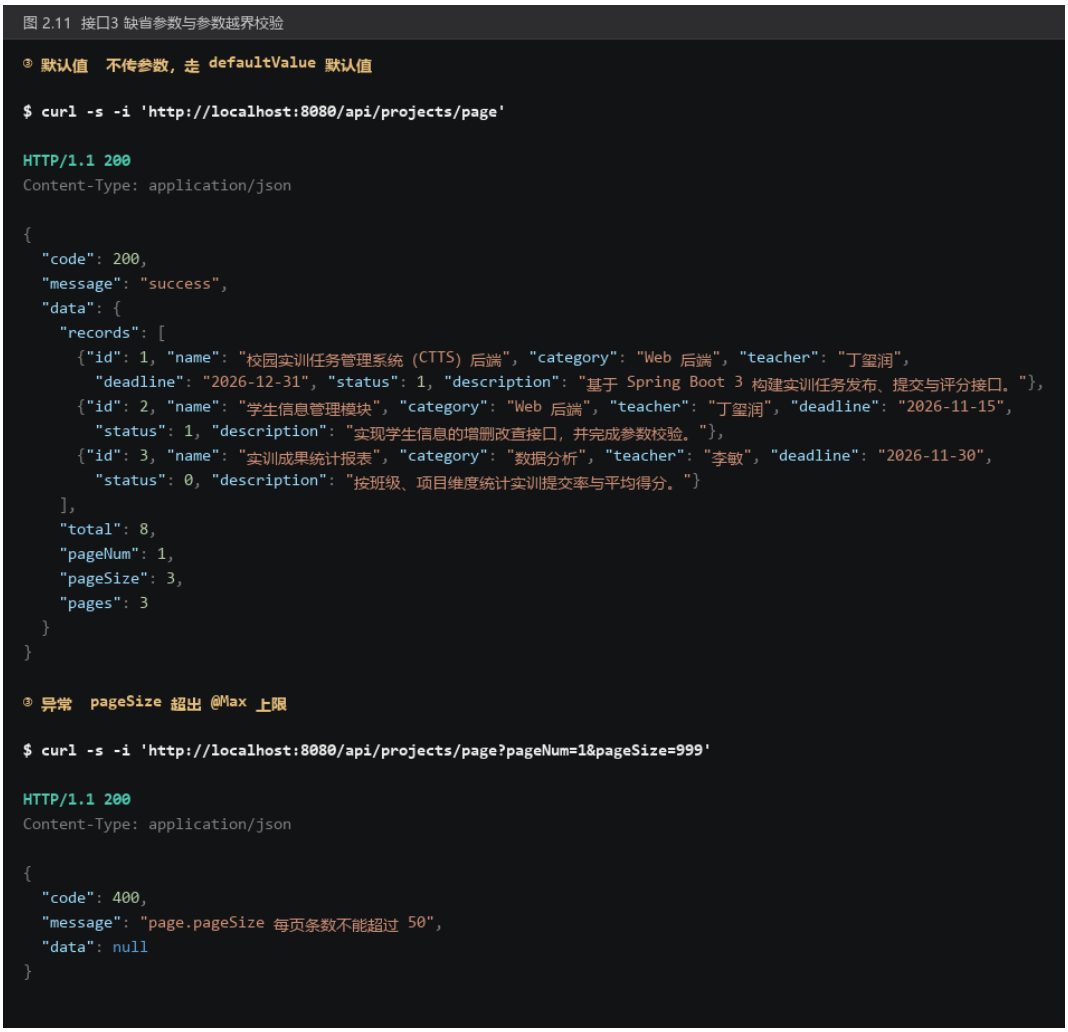


图 2.11 接口 3 缺省参数与参数越界校验

④ 新增项目。`@RequestBody` 把请求体中的 JSON 反序列化成 `ProjectForm`，`@RequestHeader` 读取请求头中的 `Authorization` 令牌，两者在同一个方法上演示了「请求体参数」与「请求头参数」。新增成功后返回生成的 id；若 `name` 为空，`@NotBlank` 校验失败，响应体中同时给出 `code=400` 与具体原因，如图 2.12 所示。

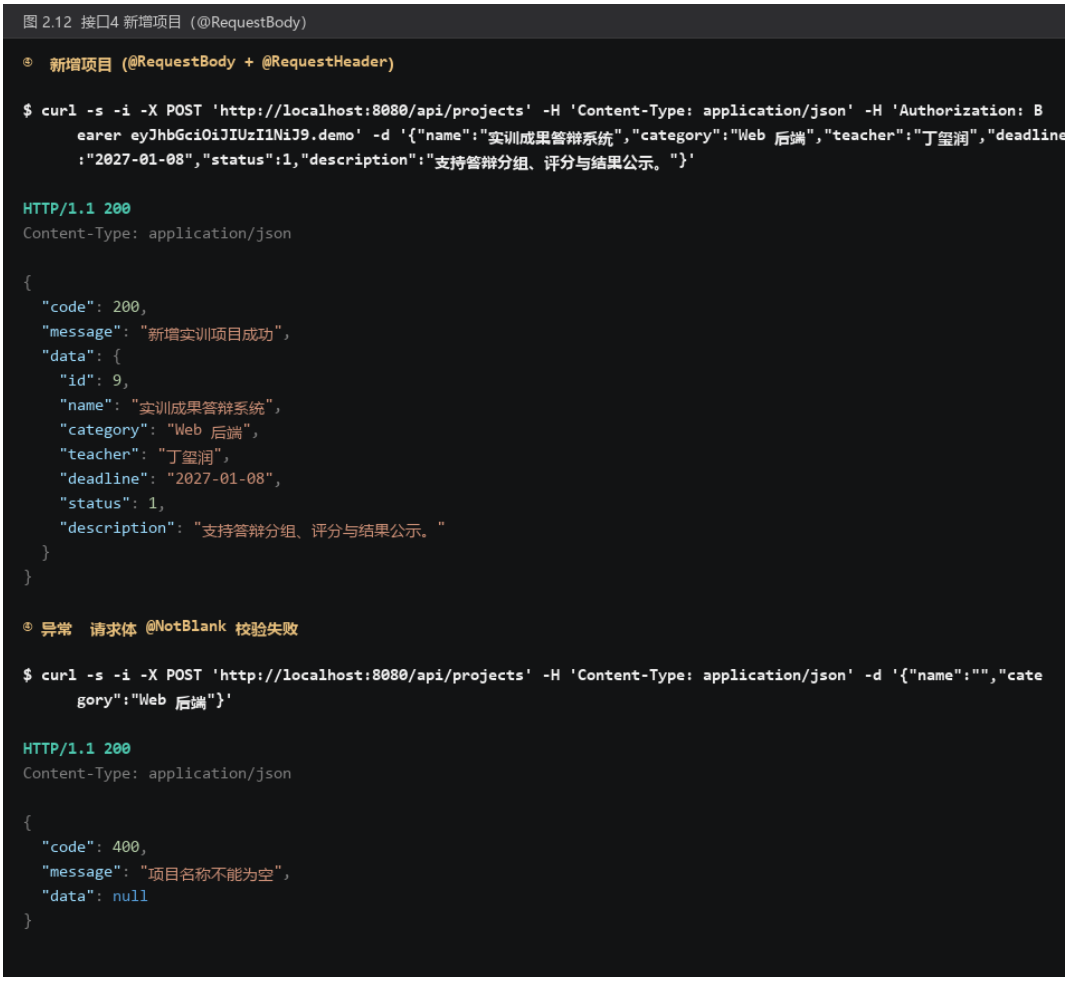


图 2.12 接口 4 新增项目（@RequestBody + @RequestHeader）

（8）对照服务端日志验证参数接收

客户端只看到响应结果，为确认参数确实按预期完成了绑定与校验，再回看服务端运行日志（图 2.13）。日志中每条 `ProjectController` 记录都打印了解析后的实际取值（如 `pageNum = 2, pageSize = 3`、`Authorization = Bearer ...`），两条 `GlobalExceptionHandler` 记录则是异常分支被统一收口的证据——二者与客户端收到的响应一一对应。

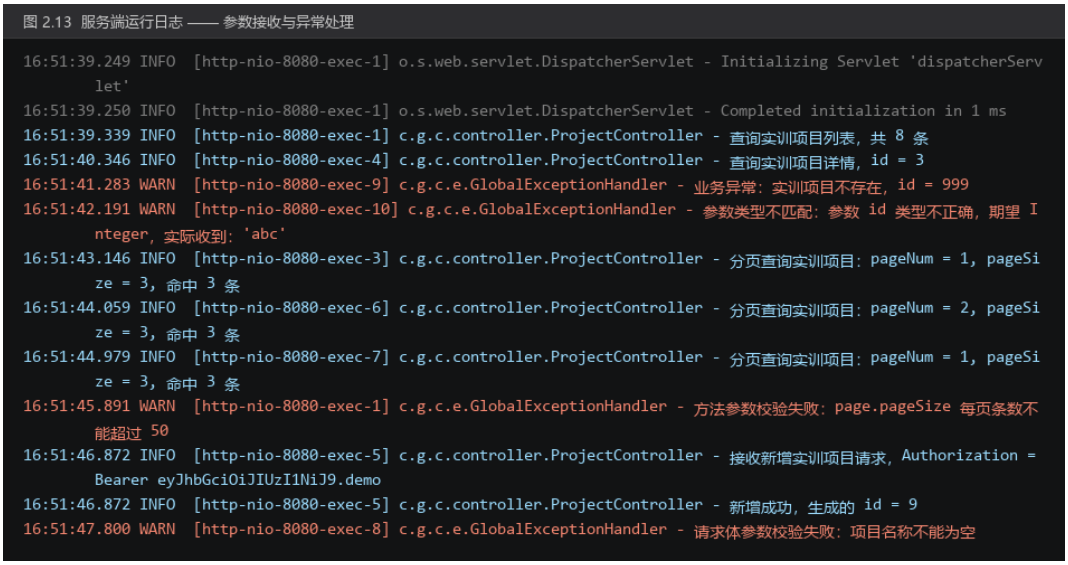


图 2.13 服务端运行日志——参数接收与异常处理

（9）调试过程中遇到的问题及解决

问题一：`GET /api/projects/list` 返回了参数类型错误而不是 404。原因是 `@GetMapping("/{id}")` 会把任何单段路径都当成 `{id}` 匹配，`list` 因此落到了接口 2 上。这说明路径变量属于「宽匹配」，设计 REST 路径时应把固定名词（如 `/page`）放在与路径变量同一层级，并让前端严格按约定调用。

问题二：`pageSize=999` 时一度返回 500。原因是初版只在 `@NotBlank` 等请求体校验上做了处理，没有覆盖方法参数校验抛出的 `ConstraintViolationException`。补充全局异常处理器后，该类异常被归一为 400，与「参数格式错误属于客户端错误」的语义保持一致。

问题三：统一响应体一度让前端无法感知失败。原因是 Controller 直接返回了业务对象，异常时由 Spring 默认错误页兜底，返回结构与其他接口不一致。引入 `Result` 后所有接口无论成功失败都是同一结构，前端只需判断 `code`。

五、实验总结：

本次实验在实验一的 CTTS 工程骨架上完成了 Controller 层的编写与联调，实现了实训项目列表、按 id 查询详情、分页查询、新增项目四个接口，并通过真实 HTTP 请求逐一验证了 @RequestParam、@PathVariable、@RequestBody 三类参数接收方式，验证结果与预期一致。主要收获有三点。其一，理解了请求映射的匹配机制。@RequestMapping 在类上提供统一前缀，方法级注解只需写相对路径；@GetMapping("/{id}") 属于宽匹配，

会把任何单段路径都吸进来，因此与路径变量同级的固定名词必须显式声明（如本实验的 /page），否则会被误匹配到详情接口上。其二，厘清了三种参数注解的分工。@RequestParam 对应 URL 查询串，适合可选的筛选与分页条件，配合 required=false 与 defaultValue 可以避免参数缺失导致的绑定失败；@PathVariable 对应 URL 路径占位符，适合表达资源标识，并会自动完成字符串到目标类型的转换；@RequestBody 对应请求体 JSON，适合结构较复杂的提交数据，必须与 Content-Type: application/json 搭配使用，否则会抛 HttpMediaTypeNotSupportedException。其三，体会到统一响应与全局异常处理的价值。引入 Result 泛型封装之后，成功与失败返回同一套三段式结构，前端只需判断 code，不必再解析 HTTP 状态码；把 BizException、MethodArgumentNotValidException、ConstraintViolationException、MethodArgumentTypeMismatchException 等异常集中到 @RestControllerAdvice 中处理，Controller 里就只剩下成功路径，代码明显清爽，也避免了默认错误页导致返回结构不一致的问题。最后，本实验仍有可改进之处：目前接口返回的是内存中的模拟数据，应用一重启数据即丢失，距离真实业务还有差距，下一步应把 Service 的实现换成 MyBatis-Plus 的数据库查询；此外本实验只做了手工调试，缺少对接口的自动化回归测试，后续可以补充 MockMvc 用例，并把参数校验规则与接口文档同步维护，降低前后端联调时的沟通成本。

六、教师评语：

评语教师：	丁奎润	实验评分：	